

Technologie Blockchain (ćwiczenia laboratoryjne nr 1)

Temat: Środowisko programistyczne PyCharm i prototypowy szkielet blockchajna

Cel ćwiczenia: Przygotowanie środowiska programistycznego PyCharm oraz zbudowanie w języku Python prototypowego szkieletu łańcucha bloków, pozwalającego zrozumieć rolę funkcji skrótu i powiązań między blokami w zapewnianiu integralności danych.

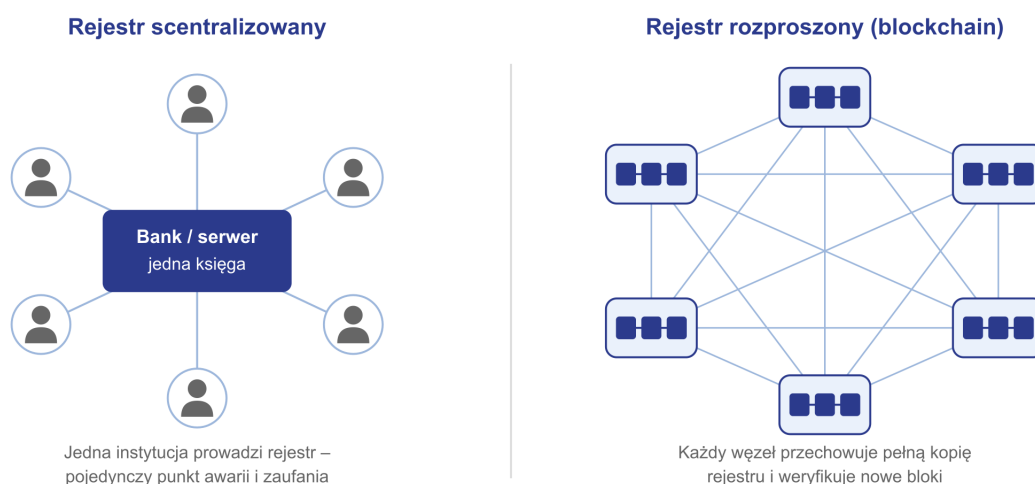
Czas realizacji: 2 godz.

1. Wprowadzenie

1.1. Czym jest blockchain?

Wyobraźmy sobie księgę, w której zapisuje się wszystkie przelewy między uczestnikami pewnej społeczności. W tradycyjnym modelu taką księgę prowadzi jedna zaufana instytucja – np. bank. To ona decyduje, które operacje są poprawne, i to jej musimy ufać, że niczego nie zmieni, nie usunie ani nie zablokuje. Jest ona również pojedynczym punktem awarii: jej niedostępność, błąd lub przejęcie przez atakującego dotyka wszystkich użytkowników.

Blockchain (łańcuch bloków) to sposób prowadzenia takiej księgi **bez centralnego administratora**. Księga jest rozproszona – pełną kopię przechowuje każdy uczestnik sieci (węzeł). Nowe wpisy są grupowane w **bloki**, które węzły samodzielnie weryfikują, a następnie – zgodnie ze wspólnymi regułami (**mechanizmem konsensusu**) – dopisują na końcu łańcucha. Raz zapisanych danych nie da się niepostrzeżenie zmienić, ponieważ każdy blok jest kryptograficznie powiązany z blokiem poprzednim.



Rys. 1. Rejestr scentralizowany i rejestr rozproszony

Idea łańcucha dokumentów powiązanych funkcjami skrótu pojawiła się już w 1991 r. (S. Haber i W. S. Stornetta, bezpieczne znakowanie czasem dokumentów cyfrowych). Przełomem był jednak dopiero **Bitcoin** (2008–2009), w którym Satoshi Nakamoto połączył łańcuch skrótów z siecią peer-to-peer i mechanizmem Proof of Work, tworząc pierwszą działającą walutę cyfrową niewymagającą zaufania do żadnej instytucji. W 2015 r. uruchomiono sieć **Ethereum**, która pozwala zapisywać w łańcuchu nie tylko transakcje, ale także programy o dowolnej złożoności, czyli inteligentne kontrakty.

Najważniejsze cechy technologii blockchain oraz mechanizmy, które je zapewniają:

Cecha	Znaczenie	Mechanizm
Decentralizacja	Brak jednego administratora i pojedynczego punktu awarii	Sieć peer-to-peer, pełne kopie rejestru
Niezmienność	Historii nie da się zmienić bez wykrycia	Funkcje skrótu, powiązania między blokami, Proof of Work
Przejrzystość	Każdy może samodzielnie zweryfikować cały rejestr	Publiczny dostęp do łańcucha
Konsensus	Uczestnicy uzgadniają jedną wersję historii	Proof of Work, Proof of Stake, BFT

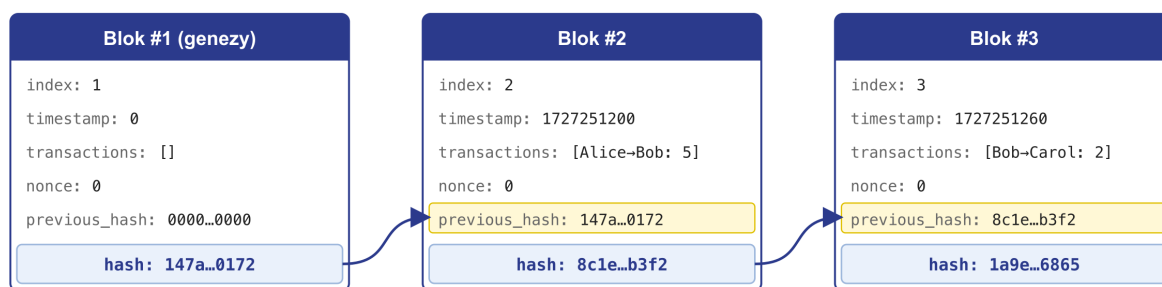
Cecha	Znaczenie	Mechanizm
Pseudonimowość	Użytkownicy są identyfikowani adresami, a nie nazwiskami	Kryptografia klucza publicznego

Ze względu na dostęp wyróżnia się blockchainya **publiczne** (każdy może czytać, wysyłać transakcje i uczestniczyć w konsensusie, np. Bitcoin, Ethereum), **prywatne** (zarządzane przez jedną organizację, np. wdrożenia Hyperledger Fabric) oraz **konsorcjalne** (zarządzane przez grupę organizacji, np. banków lub firm logistycznych). Poza kryptowalutami technologię wykorzystuje się m.in. w zdecentralizowanych finansach (DeFi), tokenizacji aktywów i NFT, śledzeniu łańcuchów dostaw, cyfrowej tożsamości oraz notaryzacji dokumentów.

Uwaga: blockchain nie jest rozwiązaniem uniwersalnym. Rozproszenie i konsensus mają swoją cenę – niską przepustowość, wysokie koszty zapisu i brak możliwości poprawiania błędów. Jeżeli wszyscy uczestnicy ufają jednemu podmiotowi, zwykła baza danych jest zwykle lepszym wyborem.

1.2. Budowa łańcucha bloków

Każdy blok zawiera skrót (hash) bloku poprzedniego, dzięki czemu jakakolwiek modyfikacja danych historycznych zmienia skróty wszystkich kolejnych bloków i jest natychmiast wykrywalna. Ta właściwość – odporność na niezauważoną modyfikację (ang. *tamper-evidence*) – jest podstawą bezpieczeństwa technologii blockchain.



Pole `previous_hash` każdego bloku zawiera skrót SHA-256 całego bloku poprzedniego
– bloki tworzą łańcuch, w którym zmiana jednego elementu „psuje” wszystkie kolejne

Rys. 2. Struktura łańcucha bloków

W trakcie ćwiczeń laboratoryjnych nr 1–4 zbudujemy krok po kroku własny, uproszczony blockchain. Każdy blok w naszym prototypie będzie słownikiem zawierającym następujące pola (Rys. 2):

- `index` – numer kolejny bloku w łańcuchu,
- `timestamp` – znacznik czasu utworzenia bloku (czas uniksowy),
- `transactions` – lista danych (transakcji) zapisanych w bloku,
- `nonce` – liczba wykorzystywana w mechanizmie Proof of Work (na tym etapie równa 0),
- `previous_hash` – skrót SHA-256 poprzedniego bloku.

Pierwszy blok łańcucha nazywany jest **blokiem genezy** (ang. *genesis block*). Nie ma on poprzednika, dlatego jego pole `previous_hash` wypełnia się umowną wartością – w naszym przypadku ciągiem 64 zer. Blok genezy ma też stały znacznik czasu, dzięki czemu będzie identyczny we wszystkich węzłach sieci.

Przykład 1: Funkcja skrótu SHA-256

Funkcja skrótu przekształca dane o dowolnej długości w ciąg o stałej długości – dla SHA-256 jest to 256 bitów, czyli 64 znaki szesnastkowe. Z punktu widzenia bezpieczeństwa kluczowe są jej właściwości: deterministyczność (te same dane dają zawsze ten sam skrót), jednokierunkowość (nie da się odtworzyć danych ze skrótu), odporność na kolizje (praktycznie niemożliwe jest znalezienie dwóch różnych danych o tym samym skrócie) oraz **efekt lawinowy** – minimalna zmiana danych wejściowych powoduje całkowitą zmianę skrótu:

```
import hashlib

print(hashlib.sha256(b"Blockchain").hexdigest())
print(hashlib.sha256(b"blockchain").hexdigest())
```

Wynik:

```
625da44e4eaf58d61cf048d168aa6f5e492dea166d8bb54ec06c30de07db57e1
ef7797e13d3a75526946a3bcf00daec9fc9c9c4d51ddc7cc5df888f74dd434d1
```

Zmiana jednej litery (B → b) dała zupełnie inny skrót. Gdyby ktoś zmienił choćby jeden znak w transakcji zapisanej w bloku, skrót tego bloku uległby całkowitej zmianie.

Przykład 2: Przygotowanie projektu w PyCharm

1. Zainstaluj interpreter Python w wersji 3.10 lub nowszej (<https://www.python.org/downloads/>) oraz środowisko PyCharm lub inne.
2. Utwórz nowy projekt. Jako interpreter wybierz nowe środowisko wirtualne (*Project venv / virtualenv*). Środowisko wirtualne izoluje biblioteki projektu od reszty systemu, dzięki czemu różne projekty mogą korzystać z różnych wersji tych samych pakietów.
3. Otwórz terminal i zainstaluj biblioteki, które będą potrzebne w kolejnych ćwiczeniach:

```
pip install flask requests
pip list
```

4. Utwórz w projekcie plik `blockchain.py`.

Przykład 3: Szkielet klasy Blockchain

Poniższy kod definiuje klasę `Blockchain`, która przechowuje łańcuch w postaci listy bloków, tworzy blok genezy i pozwala dodawać kolejne bloki:

```
import hashlib
import json
from time import time

class Blockchain:
    def __init__(self):
        self.chain = []
        self.create_genesis_block()

    def create_genesis_block(self):
        genesis = {
            "index": 1,
            "timestamp": 0,
            "transactions": [],
            "nonce": 0,
            "previous_hash": "0" * 64,
```

```

    }
    self.chain.append(genesis)

    @staticmethod
    def hash(block):
        # sort_keys=True gwarantuje deterministyczną kolejność pól
        block_string = json.dumps(block, sort_keys=True).encode()
        return hashlib.sha256(block_string).hexdigest()

    @property
    def last_block(self):
        return self.chain[-1]

    def add_block(self, transactions):
        block = {
            "index": len(self.chain) + 1,
            "timestamp": time(),
            "transactions": transactions,
            "nonce": 0,
            "previous_hash": self.hash(self.last_block),
        }
        self.chain.append(block)
        return block

if __name__ == "__main__":
    bc = Blockchain()
    bc.add_block(["Alice -> Bob: 5"])
    bc.add_block(["Bob -> Carol: 2"])
    for block in bc.chain:
        print(json.dumps(block, indent=2, ensure_ascii=False))
        print("hash:", Blockchain.hash(block))
        print("-" * 70)

```

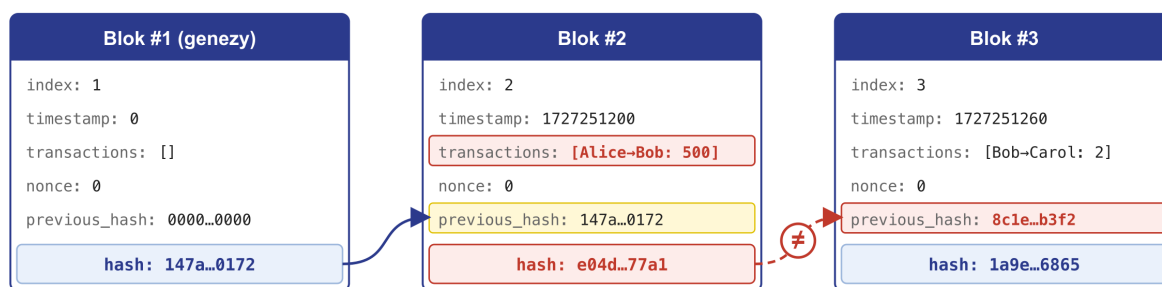
Najważniejsze elementy kodu:

- `hash()` – serializuje blok do formatu JSON i oblicza jego skrót SHA-256. Parametr `sort_keys=True` jest niezbędny: słownik o tych samych danych, ale innej kolejności kluczy, dałby inny ciąg JSON, a więc inny skrót. Metoda jest statyczna (`@staticmethod`), ponieważ nie korzysta ze stanu obiektu.
- `last_block` – właściwość (`@property`) zwracająca ostatni blok łańcucha.
- `add_block()` – tworzy nowy blok, w którym pole `previous_hash` jest skrótem aktualnie ostatniego bloku. To właśnie to pole „spina” bloki w łańcuch.

Uruchom program. Zwróć uwagę, że wartość `previous_hash` bloku nr 3 jest identyczna ze skrótem wypisanym dla bloku nr 2.

Przykład 4: Weryfikacja integralności łańcucha

Aby sprawdzić, czy łańcuch nie został zmodyfikowany, należy dla każdego bloku (poza blokiem genezy) ponownie obliczyć skrót bloku poprzedniego i porównać go z zapisanym polem `previous_hash`. Jeżeli atakujący zmieni np. kwotę transakcji w bloku nr 2, skrót tego bloku ulegnie zmianie i przestanie być zgodny z polem `previous_hash` bloku nr 3. Sytuację tę przedstawia Rys. 3.



Zmiana transakcji w bloku #2 zmienia jego skrót – pole `previous_hash` bloku #3 nie wskazuje już na aktualną zawartość bloku #2, więc manipulacja jest wykrywalna

Rys. 3. Wykrycie modyfikacji danych w bloku nr 2

Uwaga: sam łańcuch skrótów nie chroni przed atakującym, który po modyfikacji jednego bloku przeliczy i nadpisze pola `previous_hash` we wszystkich kolejnych blokach – takie przeliczenie trwa ułamek sekundy. W ćwiczeniu nr 2 wprowadzimy mechanizm Proof of Work, który czyni ten atak kosztownym obliczeniowo.

W trakcie pracy z kodem warto korzystać z debuggera PyCharm: kliknij na marginesie obok numeru linii, aby ustawić punkt przerwania (*breakpoint*), a następnie uruchom program poleceniem **Debug 'blockchain'**. W oknie *Variables* można podglądać zawartość zmiennych, a przyciskami *Step Over* / *Step Into* wykonywać kod linia po linii.

2. Zadania do zrealizowania

2.1. Przygotowanie środowiska

- Zainstaluj Python i PyCharm, utwórz projekt `blockchain-lab` z własnym środowiskiem wirtualnym.
- Zainstaluj pakiety `flask` i `requests`. Wynik polecenia `pip list` umieść w sprawozdaniu.

2.2. Implementacja szkieletu blockchajna

- Zaimplementuj klasę `Blockchain` zgodnie z Przykładem 3 i dodaj do łańcucha co najmniej 3 bloki z przykładowymi transakcjami.
- Wyświetl zawartość łańcucha wraz ze skrótami bloków i sprawdź, czy pola `previous_hash` wskazują na właściwe bloki.
- Ustaw punkt przerwania w metodzie `add_block()` i prześledź w debuggerze tworzenie nowego bloku. Zrzut ekranu okna *Variables* dołącz do sprawozdania.
- Sprawdź doświadczalnie, jak zmieni się skrót bloku, jeżeli w metodzie `hash()` usuniesz parametr `sort_keys=True` i zmienisz kolejność pól w słowniku bloku. Wyjaśnij wynik.

2.3. Weryfikacja integralności

- Zaimplementuj metodę `is_chain_valid()`, która zwraca informację, czy łańcuch jest poprawny, oraz numer pierwszego niepoprawnego bloku.
- Zmodyfikuj ręcznie kwotę transakcji w bloku nr 2 (np. `bc.chain[1]["transactions"][0] = "Alice -> Bob: 500"`) i pokaż, że metoda wykrywa manipulację.
- Następnie „napraw” łańcuch w imieniu atakującego: przelicz i nadpisz pola `previous_hash` we wszystkich blokach następujących po zmodyfikowanym. Czy metoda nadal wykrywa atak? Wyjaśnij, dlaczego.
- Zmodyfikuj ostatni blok. Czy metoda to wykryje? Dlaczego?

2.4. Sprawozdanie

- Przygotuj sprawozdanie zawierające: kod źródłowy (lub link do repozytorium), zrzuty ekranu potwierdzające wykonanie poszczególnych zadań, wyniki pomiarów oraz odpowiedzi na pytania postawione w zadaniach i pytania kontrolne.

3. Pytania kontrolne

1. Jakie właściwości funkcji skrótu decydują o bezpieczeństwie łańcucha bloków?
2. Dlaczego przy obliczaniu skrótu bloku stosuje się serializację z posortowanymi kluczami?
3. Czym różni się blok genezy od pozostałych bloków i dlaczego musi być identyczny we wszystkich węzłach?
4. Dlaczego sam łańcuch skrótów nie chroni przed atakującym, który przeliczy wszystkie kolejne bloki?